

2. 矩阵运算

- 2.1 矩阵的加减法
- 2.2 矩阵的数乘
- 2.3 矩阵的乘积
 - 矩阵乘法 $\times$
 - Hadamard 积 $\odot$ 、Kronecker 积 $\otimes$ 与外积
- 2.4 矩阵的转置
- 2.5 矩阵求逆
- 2.6 伴随矩阵

参考资料:

- [1] [Mathematics for Deep Learning: Matrix, Matrices Addition and Matrices Multiplication](#)
- [2] [Intuitively Understanding Convolutions for Deep Learning](#)
- [3] [A Gentle Introduction to Matrix Operations for Machine Learning](#)



2. 矩阵运算

• 2.1 矩阵的加减法

设有两个 $m \times n$ 的矩阵 $A = (a_{ij}), B = (b_{ij})$

那么矩阵A与矩阵B的和矩阵记作 $A+B$, 规定为:

$$A+B = \begin{pmatrix} a_{11}+b_{11} & a_{12}+b_{12} & \Lambda & a_{1n}+b_{1n} \\ a_{21}+b_{21} & a_{22}+b_{22} & \Lambda & a_{2n}+b_{2n} \\ \text{M} & \text{M} & & \text{M} \\ a_{m1}+b_{m1} & a_{m2}+b_{m2} & \Lambda & a_{mn}+b_{mn} \end{pmatrix}$$

对应位置上的元素相加

注意：只有同型矩阵才能相加



2. 矩阵运算

• 2.1 矩阵的加减法

- 矩阵加法满足以下运算规律：

(i) $A+B=B+A$ (交换律)

(ii) $(A+B)+C=A+(B+C)$ (结合律)

(iii) $A+O=O+A=A$

- 设矩阵 $A = (a_{ij})$, 则 $-A = (-a_{ij})$, $-A$ 称为矩阵A的负矩阵, 显然有

$$A+(-A)=(-A)+A=O$$

定义矩阵的减法: $A-B=A+(-B)$



2. 矩阵运算

• 2.1 矩阵的加减法

- 矩阵加减运算在机器学习中的应用示例:

- 数据预处理与标准化

- 数据中心化 (均值减法) : $X_{\text{centered}} = X - \mathbf{1}_n \mu^\top$

- 标准化 (Standardization) : $X_{\text{std}} = \frac{X - \mathbf{1}_n \mu^\top}{\mathbf{1}_n \sigma^\top}$

- 模型训练与参数更新

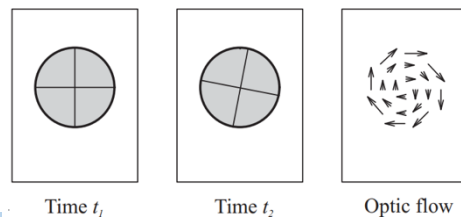
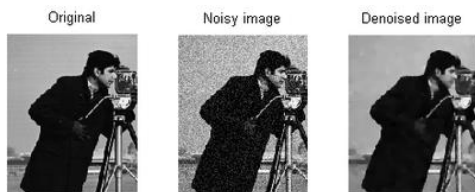
- 梯度下降优化: $w \leftarrow w - \eta \cdot \nabla_w \mathcal{L}$

- 残差网络 (ResNet) 通过Skip Connection缓解深层网络梯度消失问题 $y = F(x) + x$

- 图像与信号处理

- 图像增强与噪声去除 $v(x) = u(x) + \eta(x), \quad x \in \Omega,$

- 运动检测 $f(x, y, t) = f(x + dx, y + dy, t + dt)$



参考资料:

[1] [Introduction To Image Denoising](#)

[2] [Image motion](#)

2. 矩阵运算

• 2.2 矩阵的数乘

- 矩阵数乘是指将一个标量（实数或复数）与矩阵的每个元素相乘的运算。设标量为 a ，矩阵 $A = [a_{ij}] \in \mathbb{R}^{m \times n}$ ，则数乘结果为：

$$\alpha A = [\alpha \cdot a_{ij}]_{m \times n}$$

运算规则：

$$\begin{aligned} (1) \quad (\lambda\mu)A &= \lambda(\mu A) & (2) \quad (\lambda + \mu)A &= \lambda A + \mu A \\ (3) \quad \lambda(A + B) &= \lambda A + \lambda B & (4) \quad 1 \cdot A &= A \\ (5) \quad (-1) \cdot A &= -A & (6) \quad 0 \cdot A &= O \end{aligned}$$

• 应用

- 图像处理：图像亮度调整
- 数据预处理：特征缩放、归一化
- 优化算法：学习率调整、梯度聚合
- 网络结构设计：激活函数、批量归一化
-



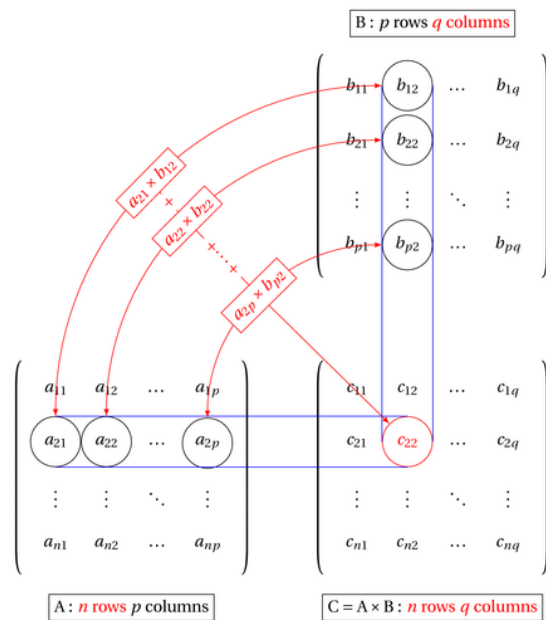
2. 矩阵运算

• 2.3 矩阵的乘法

设矩阵 $A=(a_{ij})_{m \times l}$ 的列数与矩阵 $B=(b_{ij})_{l \times n}$ 的行数相等, 则由元素

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \dots + a_{il}b_{lj} = \sum_{k=1}^l a_{ik}b_{kj}$$
$$(i = 1, 2, \dots, m; j = 1, 2, \dots, n),$$

构成的 $m \times n$ 矩阵 $C=(c_{ij})_{m \times n}$ 称为矩阵A与矩阵B的乘积, 记作 $C=AB$



2. 矩阵运算

• 2.3 矩阵的乘法

- 矩阵乘法运算的注意事项：

(1) 两矩阵相乘时，前矩阵（居左）每一行（如第*i*行）的各元素与后矩阵（居右）每一列（如第*j*列）中顺次对应的各元素相乘再相加，从而得到乘积矩阵（第*i*行第*j*列）的元素。

$$A = \begin{pmatrix} a_{11} & \text{K} & a_{1l} \\ \text{M} & \textcolor{red}{i \text{ 行}} & \text{M} \\ a_{m1} & \text{L} & a_{ml} \end{pmatrix}_{m \times l} \quad B = \begin{pmatrix} b_{11} & \text{K} & b_{1n} \\ \text{M} & \textcolor{red}{j \text{ 列}} & \text{M} \\ b_{l1} & \text{L} & b_{ln} \end{pmatrix}_{l \times n}$$

(2) 为保证规则（1），左矩阵的列数应与右矩阵的行数相等，否则两矩阵不能相乘。

(3) 乘积矩阵的行数与左矩阵相同，乘积矩阵的列数与右矩阵相同。



2. 矩阵运算

• 2.3 矩阵的乘法

设两个商店销售三种电视机的
数量（百台）由矩阵A表示

$$A = \begin{pmatrix} 12 & 8 & 10 \\ 14 & 9 & 6 \end{pmatrix} \begin{matrix} \text{百佳} \\ \text{华润} \end{matrix}$$

长虹	康佳	创维
----	----	----

三种电视机的零售单价
（千元）由矩阵B表示

$$B = \begin{pmatrix} 2.5 \\ 3 \\ 3.5 \end{pmatrix} \begin{matrix} \text{长虹} \\ \text{康佳} \\ \text{创维} \end{matrix}$$

则两商场销售电视机所得收益分别是多少？

2. 矩阵运算

• 2.3 矩阵的乘法

设两个商店销售三种电视机的
数量（百台）由矩阵A表示

三种电视机的零售单价
（千元）由矩阵B表示

$$A = \begin{pmatrix} 12 & 8 & 10 \\ 14 & 9 & 6 \end{pmatrix} \begin{matrix} \text{百佳} \\ \text{华润} \\ \begin{matrix} \text{长虹} \\ \text{康佳} \\ \text{创维} \end{matrix} \end{matrix}$$

$$B = \begin{pmatrix} 2.5 \\ 3 \\ 3.5 \end{pmatrix} \begin{matrix} \text{长虹} \\ \text{康佳} \\ \text{创维} \end{matrix}$$

则两商场销售电视机所得收益分别是多少？

$$A = \begin{pmatrix} 12 & 8 & 10 \\ 14 & 9 & 6 \end{pmatrix} \quad B = \begin{pmatrix} 2.5 \\ 3 \\ 3.5 \end{pmatrix}$$

$$\begin{aligned} C = AB &= \begin{pmatrix} 12 & 8 & 10 \\ 14 & 9 & 6 \end{pmatrix} \begin{pmatrix} 2.5 \\ 3 \\ 3.5 \end{pmatrix} \\ &= \begin{pmatrix} 12 \times 2.5 + 8 \times 3 + 10 \times 3.5 \\ 14 \times 2.5 + 9 \times 3 + 6 \times 3.5 \end{pmatrix} = \begin{pmatrix} 89 \\ 83 \end{pmatrix} \end{aligned}$$



2. 矩阵运算

• 2.3 矩阵的乘法

- Hadamard 积 (逐元素乘法, Element-wise Product)

- 定义: 设 $A, B \in \mathbb{R}^{m \times n}$, Hadamard 积定义为同形矩阵的逐元素相乘:

$$(A \odot B)_{ij} = A_{ij}B_{ij}$$

⚠ 与矩阵乘法的区别

- 普通乘法 AB : 要求 A 为 $m \times k$, B 为 $k \times n$, 结果为 $m \times n$ (行列数改变)
- Hadamard 积 $A \odot B$: 要求 A 、 B 形状完全相同, 结果形状不变
- NumPy/PyTorch 中: $A @ B$ 是矩阵乘法; $A * B$ 是 Hadamard 积

应用场景	数学表达	说明
激活函数逐元素作用	$Z = \sigma(WX + b)$, 对每个元素独立作用	ReLU, Sigmoid, Tanh 均为 Hadamard 操作
LSTM 门控机制	$c_t = f_t \odot c_{t-1} + i_t \odot g_t$	遗忘门和输入门逐元素控制记忆
Dropout 正则化	$h = \text{mask} \odot x$ ($\text{mask} \in \{0, 1\}^n$)	随机置零部分神经元输出
Attention 权重掩码	$\text{Score} = \text{Score} \odot \text{Mask}$ ($-\infty$ 填充)	Transformer中屏蔽无效位置
批归一化 (BN)	$y = \gamma \odot \hat{x} + \beta$ (逐通道缩放)	γ, β 为可学习的逐元素参数

2. 矩阵运算

• 2.3 矩阵的乘法

- Kronecker 积（张量积）→ 空间爆炸/维度扩张

- 定义：设 $A \in \mathbb{R}^{m \times n}$, $B \in \mathbb{R}^{p \times q}$, Kronecker 积 $A \otimes B \in \mathbb{R}^{mp \times nq}$ 定义为：

$$A \otimes B = \begin{bmatrix} a_{11}B & a_{12}B & \dots & a_{1n}B \\ a_{21}B & a_{22}B & \dots & a_{2n}B \\ \dots & \dots & \dots & \dots \end{bmatrix}$$

- 将 A 的每个元素替换为它与 B 的数乘
- 结果是一个分块矩阵，每块是 $A_{ij} \cdot B$

- 示例： $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $B = \begin{bmatrix} 0 & 5 \\ 6 & 7 \end{bmatrix}$, 则 $A \otimes B$ 为 4×4 矩阵：

$$A \otimes B = \begin{bmatrix} 1 \cdot B & 2 \cdot B \\ 3 \cdot B & 4 \cdot B \end{bmatrix} = \begin{bmatrix} 0 & 5 & 0 & 10 \\ 6 & 7 & 12 & 14 \end{bmatrix}$$



2. 矩阵运算

• 2.3 矩阵的乘法

• Kronecker 积（张量积）

• 重要性质：

混合积性质	$(A \otimes B)(C \otimes D) = (AC) \otimes (BD)$ （当维度匹配时）
逆与转置	$(A \otimes B)^{-1} = A^{-1} \otimes B^{-1}$ ； $(A \otimes B)^T = A^T \otimes B^T$

• 应用示例：

AI应用场景	如何用到 Kronecker 积	意义
多任务学习中的结构化正则化	$W = W_1 \otimes W_2$ （分解参数矩阵）	减少参数量，引入任务间共享结构
图卷积网络（GCN）	谱域卷积可表示为 Kronecker 积形式	高效批量图操作



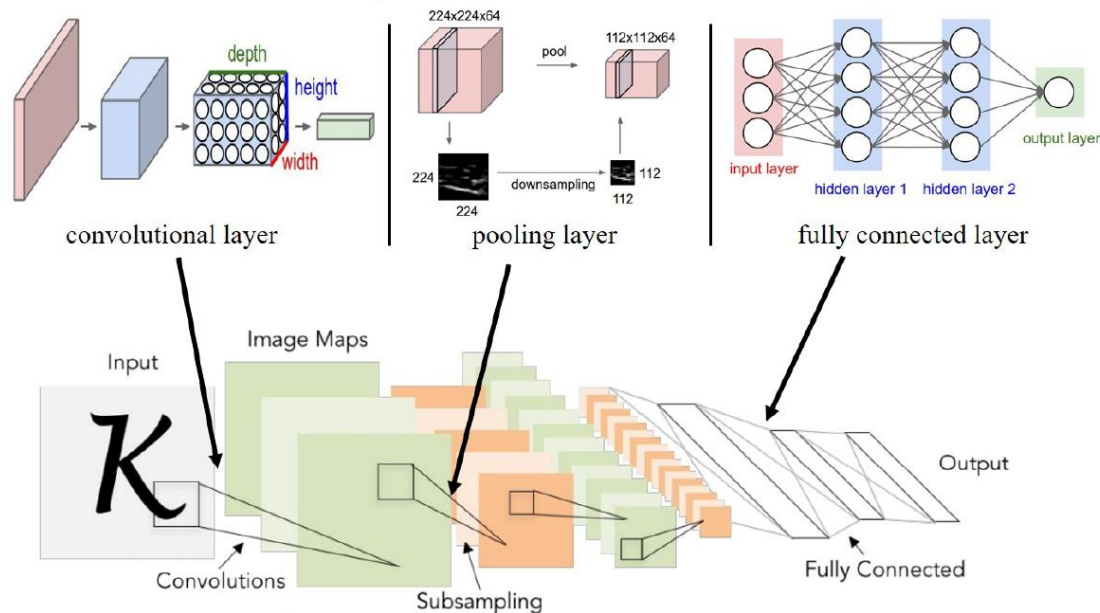
2. 矩阵运算

• 2.3 矩阵的乘法

• 矩阵乘法在机器学习中的应用示例：

- ▶ A Convolutional Neural Network (CNN) is comprised of one or more **convolutional layers** (often with a **subsampling step**) and then followed by one or more **fully connected layers** as in a standard multilayer neural network.

- ▶ Convolutional Layer
- ▶ Pooling Layer
- ▶ Fully-Connected Layer(exactly as seen in regular Neural Networks)



The first successful applications of Convolutional Networks: LeNet-5



2. 矩阵运算

• 2.3 矩阵的乘法

- 矩阵乘法在机器学习中的应用示例：

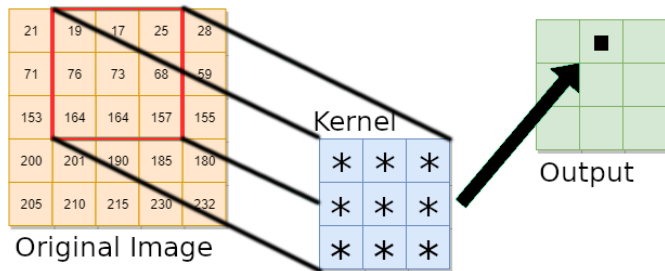
- 图像处理：卷积、边缘检测等



Original Image

Image filtered
with matrix

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



Original	Gaussian Blur	Sharpen	Edge Detection
$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$

Kernel types

参考资料：

- [1] [How Image Edge Detection Works](#)
- [2] [Understanding “convolution” operations in CNN](#)
- [3] [Why GEMM is at the heart of deep learning](#)

2. 矩阵运算

• 2.3 矩阵的乘法

• 卷积操作和矩阵乘法

- **卷积操作是一种特殊的矩阵乘法**：在卷积神经网络 (CNN) 等模型中，卷积操作可以看作是一种特殊的矩阵乘法。在卷积操作中，卷积核（或滤波器）可以视作一个小的矩阵，与输入图像的像素矩阵进行卷积运算，得到输出图像的像素值。这个过程涉及到对输入图像的像素矩阵和卷积核矩阵进行乘法运算，并将结果相加以生成输出图像的像素值。
- **矩阵乘法是一种全连接操作，而卷积操作是局部连接的**：在矩阵乘法中，每个输入元素都与权重矩阵中的所有权重进行乘法运算，从而得到输出向量。而在卷积操作中，每个输出元素仅与输入图像中的局部区域进行卷积运算，这种局部连接的方式可以减少参数数量和计算量，提高模型的效率。
- **卷积操作具有平移不变性，而矩阵乘法不具备**：在卷积操作中，卷积核与输入图像进行卷积运算时具有平移不变性，即无论目标特征在图像中的位置如何，都可以被卷积核正确地识别。而在矩阵乘法中，每个输入元素与权重矩阵中的对应权重相乘，位置信息不会被保留，因此矩阵乘法不具备平移不变性。
- 卷积操作本质上不属于标准矩阵乘法，但在数学上可以通过矩阵化（Matrixization）技巧转化为矩阵乘法的形式，从而利用矩阵运算的优化工具（如 BLAS 库）加速计算。



2. 矩阵运算

• 2.3 矩阵的乘法

• 矩阵乘法在机器学习中的应用示例：

• 卷积运算

核心思想：将输入矩阵的每个局部窗口展开为列向量，卷积核展平为行向量，通过矩阵乘法实现卷积。

步骤：

1. 输入矩阵展开 (Im2col)

设输入为 $X \in \mathbb{R}^{H \times W}$ ，卷积核为 $K \in \mathbb{R}^{k \times k}$ ，步长为 s 。将每个 $k \times k$ 的滑动窗口按行展平为长度 k^2 的向量，生成矩阵 $X_{\text{col}} \in \mathbb{R}^{N \times k^2}$ ，其中 $N = \frac{(H-k)}{s+1} \cdot \frac{(W-k)}{s+1}$ 是窗口总数。

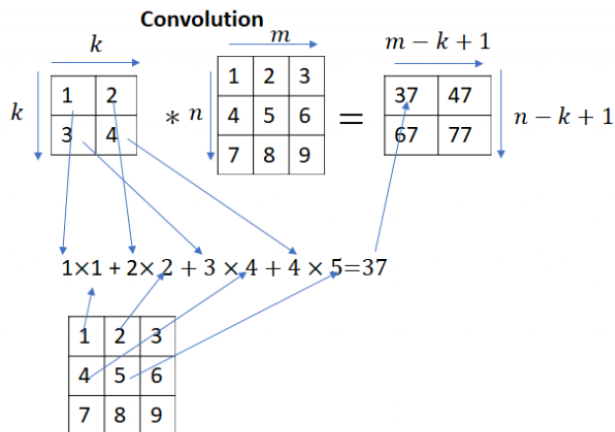
2. 卷积核向量化

将卷积核 K 展平为行向量 $K_{\text{vec}} \in \mathbb{R}^{1 \times k^2}$ 。

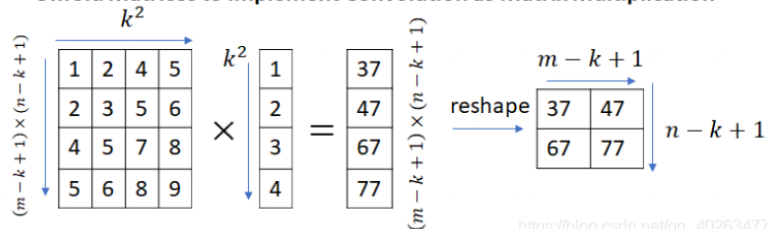
3. 矩阵乘法

卷积结果等价于： $Y_{\text{vec}} = K_{\text{vec}} \cdot X_{\text{col}}^T$ 或 $Y_{\text{vec}}^T = X_{\text{col}} \cdot K_{\text{vec}}^T$

其中 $Y_{\text{vec}} \in \mathbb{R}^{1 \times N}$ 是卷积结果的向量化形式，需重塑为 $\mathbb{R}^{\frac{(H-k)}{s+1} \times \frac{(W-k)}{s+1}}$ 。



Unfold Matrices to Implement Convolution as Matrix Multiplication



https://blog.csdn.net/qq_40263477



2. 矩阵运算

• 2.3 矩阵的乘法

• 卷积操作和矩阵乘法

- 将卷积转换为矩阵乘法的本质是**用空间换时间**，通过牺牲一定内存换取计算效率的大幅提升，同时复用成熟的线性代数工具链。
- 这一转换是深度学习工程化的关键技术之一，尤其在 GPU 加速和大规模模型训练中不可或缺。尽管存在内存开销等局限，但其在硬件兼容性和计算效率上的优势使其成为长期沿用的经典方法。



2. 矩阵运算

• 2.3 矩阵的乘法

例： 设 $A = \begin{pmatrix} 1 & 2 \\ 4 & 0 \\ 3 & 5 \end{pmatrix}$ $B = \begin{pmatrix} 3 & 1 \\ 2 & 4 \end{pmatrix}$, 求 AB , $A \odot B$, $A \otimes B$ 。

解

$$AB = \begin{pmatrix} 1 & 2 \\ 4 & 0 \\ 3 & 5 \end{pmatrix} \begin{pmatrix} 3 & 1 \\ 2 & 4 \end{pmatrix}$$
$$= \begin{pmatrix} 1 \times 3 + 2 \times 2 & 1 \times 1 + 2 \times 4 \\ 4 \times 3 + 0 \times 2 & 4 \times 1 + 0 \times 4 \\ 3 \times 3 + 5 \times 2 & 3 \times 1 + 5 \times 4 \end{pmatrix} = \begin{pmatrix} 7 & 9 \\ 12 & 4 \\ 19 & 23 \end{pmatrix}$$

矩阵与矩阵相乘不满足交换律， AB 有意义，但 BA 不一定有意义



2. 矩阵运算

• 2.3 矩阵的乘法

例： $A = \begin{pmatrix} 0 & 3 \\ 2 & 0 \end{pmatrix}$ $B = \begin{pmatrix} -9 & 6 \\ 4 & -9 \end{pmatrix}$ 求AB和BA

解： $AB = \begin{pmatrix} 0 & 3 \\ 2 & 0 \end{pmatrix} \begin{pmatrix} -9 & 6 \\ 4 & -9 \end{pmatrix} = \begin{pmatrix} 12 & -27 \\ -18 & 12 \end{pmatrix}$

$$BA = \begin{pmatrix} -9 & 6 \\ 4 & -9 \end{pmatrix} \begin{pmatrix} 0 & 3 \\ 2 & 0 \end{pmatrix} = \begin{pmatrix} 12 & -27 \\ -18 & 12 \end{pmatrix}$$

$$AB = BA$$

如果同阶方阵A和B满足 $AB=BA$,则称A与B可交换



2. 矩阵运算

• 2.3 矩阵的乘法

例: $A = \begin{pmatrix} -2 & 4 \\ 1 & -2 \end{pmatrix} \quad B = \begin{pmatrix} 2 & 4 \\ -3 & -6 \end{pmatrix} \quad C = \begin{pmatrix} -4 & 8 \\ 2 & -4 \end{pmatrix}$

求AB、BA和BC

解 $AB = \begin{pmatrix} -16 & -32 \\ 8 & 16 \end{pmatrix} \quad BA = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$

$$BC = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$$

1) AB与BA都有意义, 且同型, 但AB与BA不相等

2) 两个非零矩阵相乘可能是零矩阵

3) BA=BC, 但A≠C, 可见, 矩阵乘法不满足消去率

→ $AB \neq BA, BA=BC$



2. 矩阵运算

• 2.3 矩阵的乘法

- 矩阵的乘法不满足交换律、消去律
- 但满足下列运算：

1. $(AB)C = A(BC)$

2. $\lambda(AB) = (\lambda A)B = A(\lambda B)$

3. $A(B + C) = AB + AC$
 $(B + C)A = BA + CA$

4. $I_m A_{m \times n} = A_{m \times n}$, $A_{m \times n} I_n = A_{m \times n}$



2. 矩阵运算

• 2.3 矩阵的乘法

• 矩阵的幂

设 A 是 n 阶方阵, k 为正整数, 则 A^k 表示 k 个 A 连乘, 如

$$A^3 = AAA$$

显然, 只有方阵的幂才有意义

$$A^k A^l = A^{k+l} \quad (A^k)^l = A^{kl}$$

$$(AB)^k \neq A^k B^k$$



2. 矩阵运算

• 2.3 矩阵的乘法

- 传统代数运算中的矩阵乘法同机器学习中的矩阵乘法的区别
 - 机器学习中的矩阵乘法是传统数学定义的**工程化延伸**，更注重处理大规模数据的效率、数值稳定性和与优化流程的集成，而非纯粹的数学推导。这种差异本质上是由“理论研究”与“应用落地”的不同目标导致的。

维度	传统代数矩阵乘法	机器学习矩阵乘法
数据维度	二维矩阵	二维 / 多维张量（支持批次、通道等）
数据类型	精确数值 / 符号变量	单 / 双精度浮点数（含近似误差）
运算目标	线性变换 / 数学求解	优化驱动的复合操作（如分类、回归）
计算优化	基于数学性质（如对称性）	依赖硬件加速（GPU/TPU）与底层算法
伴随操作	无	自动微分 / 反向传播
特殊变体	严格遵循定义	支持广播、分组、稀疏等扩展操作

2. 矩阵运算

• 2.4 矩阵的转置 (Transpose)

- **定义：**把矩阵A的行换成同序数的列得到的一个新矩阵，称为A的转置矩阵，记作 A^T 或 A'

$$A = \begin{pmatrix} 1 & 2 & 0 \\ 3 & -1 & 1 \end{pmatrix} \quad A^T = \begin{pmatrix} 1 & 3 \\ 2 & -1 \\ 0 & 1 \end{pmatrix}$$

运算律

$$(A^T)^T = A$$

$$(A+B)^T = A^T + B^T$$

$$(\lambda A)^T = \lambda A^T$$

$$(AB)^T = B^T A^T \Rightarrow (A_1 A_2 \text{L} A_n)^T = A_n^T \text{L} A_2^T A_1^T$$



2. 矩阵运算

• 2.4 矩阵的转置 (Transpose)

- 对称矩阵 (Symmetric matrix or Hermitian matrix):

- 如果方阵A满足 $A^T = A$, 则称A为对称矩阵

$$\begin{pmatrix} 10 & 1 \\ 1 & -1 \end{pmatrix} \quad \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix} \quad \begin{pmatrix} -3 & 2 & -4 \\ 2 & 0 & 7 \\ -4 & 7 & 5 \end{pmatrix}$$

方阵A为对称矩阵 $\iff$ 矩阵A中关于主对角线对称位置上的每一对元素都相等

$$a_{ij} = a_{ji}$$



2. 矩阵运算

• 2.4 矩阵的转置 (Transpose)

• 对称矩阵在机器学习中有着重要应用：

- **PCA**中协方差矩阵 Σ ：协方差矩阵 $\Sigma = \frac{1}{n-1} \mathbf{X}^T \mathbf{X}$ ($\mathbf{x}$ 为中心化后的数据矩阵)
 - $\Sigma^T = \Sigma$ ，对称矩阵保证可正交对角化，然后利用矩阵分解简化运算
- **核矩阵**：在核方法中，对称矩阵被广泛应用。核矩阵是由样本数据之间的核函数计算得到的对称矩阵 $K_{ij} = k(x_i, x_j)$, k 为核函数，它能够捕捉样本之间的非线性关系。在支持向量机 (SVM) 等算法中，核矩阵用于将数据从原始空间映射到高维特征空间，从而实现数据的非线性分类和回归。
- **相似度矩阵**：在聚类 and 相似度学习中，对称矩阵被用来表示样本之间的相似度或距离。通过计算样本之间的相似度，可以构建一个对称的相似度矩阵，进而用于聚类分析、流形学习等任务。
- **拉普拉斯矩阵**：在图论和图数据挖掘中，对称矩阵常被用来表示图结构。拉普拉斯矩阵是一种常见的对称矩阵，它用于描述图的拓扑结构和节点之间的关系。在谱聚类算法中，拉普拉斯矩阵被用来实现图的划分和聚类。



2. 矩阵运算

e.g. $\underbrace{\begin{bmatrix} 3 & 4 \\ 2 & 16 \end{bmatrix}}_A \underbrace{\begin{bmatrix} 0.4 & -0.1 \\ -0.05 & 0.075 \end{bmatrix}}_{A^{-1}} = \underbrace{\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}}_{A^{-1}A} = I_{2 \times 2}$

• 2.5 矩阵求逆 (Inverse)

- 定义：设A为n阶方阵，如果存在n阶方阵B，使得

$$AB=BA=I$$

就称为A可逆矩阵，并称B为A的逆矩阵（简称A的逆），记作

$$B = A^{-1}$$

- 性质1：**如果A是可逆矩阵(非奇异矩阵)，则A的逆矩阵是唯一的。

证明： 设B和C都是A的逆矩阵，则 $AB=BA=I$, $AC=CA=I$

$$B = BI = B(AC) = (BA)C = IC = C$$



2. 矩阵运算

• 2.5 矩阵求逆 (Inverse)

- **性质2:** 如果矩阵A可逆, 则 $AB=I$ 等价于 $BA=I$ 。

证明 (1) $AB = I \Rightarrow BA = I$

$$BA = I(BA) = (A^{-1}A)(BA) = A^{-1}(AB)A$$

$$= A^{-1}IA = A^{-1}A = I$$

相似可证 $BA = I \Rightarrow AB = I$

- **性质3:**
 - (1) 若A可逆, $\lambda \neq 0$, 则 $(\lambda A)^{-1} = \lambda^{-1}A^{-1}$
 - (2) 若A可逆, 则 $(A^T)^{-1} = (A^{-1})^T$
 - (3) 如果A为可逆矩阵, 则 A^{-1} 也可逆, 且 $(A^{-1})^{-1} = A$



2. 矩阵运算

• 2.5 矩阵求逆 (Inverse)

- **性质4:** 如果A和B为同阶可逆矩阵, 则AB可逆, 且

$$(AB)^{-1} = B^{-1}A^{-1}$$

证明

$$(AB)(B^{-1}A^{-1}) = A(BB^{-1})A^{-1} = AIA^{-1} = AA^{-1} = I$$

故由推论1便知AB可逆, 且 $B^{-1}A^{-1} = (AB)^{-1}$

$$\Rightarrow (A_1A_2L A_n)^{-1} = A_n^{-1}L A_2^{-1}A_1^{-1}$$



2. 矩阵运算

• 2.5 矩阵求逆 (Inverse)

- 方阵的可逆满足性质

$$(1) (A^{-1})^{-1} = A; \quad (2) (kA)^{-1} = \frac{1}{k} A^{-1} \quad (k \neq 0);$$

$$(3) A、B \text{ 均是同阶可逆阵, 则 } (AB)^{-1} = B^{-1}A^{-1};$$

$$(4) (A^T)^{-1} = (A^{-1})^T.$$

证明

$$(3) (AB)(B^{-1}A^{-1}) = A(BB^{-1})A^{-1} = AEA^{-1} = AA^{-1} = E.$$

$$(4) A^T(A^{-1})^T = (A^{-1}A)^T = (E)^T = E,$$



2. 矩阵运算

• 2.5 矩阵求逆 (Inverse)

- 利用增广矩阵求可逆矩阵 (手工计算, 也称为高斯-约当消元法)
 - 单位矩阵 I_n 可通过初等行变换得到初等矩阵 A_n , 反之, 初等矩阵 A_n 也可以通过逆变换得到单位矩阵 I_n , 该逆变换所对应的矩阵即为 A_n 的逆矩阵
 - 例:

- 单位矩阵 I_n 通过初等行变换 ($r_3 \leftarrow r_3 + r_1 \cdot (-4)$) 得到 $A_n = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -4 & 0 & 1 \end{bmatrix}$
- 将 A_n 进行初等行变换得到单位矩阵 I_n , 同形状的单位矩阵也同时变换

增广矩阵 $\left(\begin{array}{ccc|ccc} 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ -4 & 0 & 1 & 0 & 0 & 1 \end{array} \right)$ 进行初等行变换 ($r_3 \leftarrow r_3 + r_1 \cdot 4$), 得到:

$$\left(\begin{array}{ccc|ccc} 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 4 & 0 & 1 \end{array} \right), \text{ 则 } A^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 4 & 0 & 1 \end{bmatrix}$$



2. 矩阵运算

• 2.5 矩阵求逆 (Inverse)

- 矩阵求逆的工程视角
 - 求解线性方程组 $Ax = b$ 的答案是 $x = A^{-1}b$
 - 是否可以通过 `x = torch.inverse(A) @ b` 直接计算？

AI 工程界的三大潜规则：

- **真实世界的数据，通常是“不可逆”的（奇异的），** 因为数据充满了冗余和共线性，导致矩阵行列式为 0。
- **计算极其昂贵：** 求逆的时间复杂度是 $O(N^3)$ ，如果 A 是一个 10000×10000 的矩阵，求逆会让机器算到崩溃。
- **数值极不稳定：** 如果矩阵中存在非常小的数字，求逆后会被放大成极其庞大的数字（浮点数灾难），这就是**病态矩阵 (Ill-conditioned Matrix)**。

在工程应用中避免直接求逆矩阵！



2. 矩阵运算

• 2.5 矩阵求逆 (Inverse)

- LU 分解法:

- LU 分解将矩阵分解为一个下三角矩阵 (L) 和一个上三角矩阵 (U) 的乘积, 然后通过分别求解 (L) 和 (U) 的逆来得到原矩阵的逆。LU 分解的复杂度一般也为 ($O(n^3)$), 但在某些情况下比高斯-约当消元法更高效。

- Cholesky 分解法:

- Cholesky 分解是针对对称正定矩阵的一种分解方法, 将矩阵分解为一个下三角矩阵和其转置矩阵的乘积。对于对称正定矩阵, Cholesky 分解比 LU 分解更快, 并且更稳定。

- 迭代法:

- 迭代法是一种近似求解矩阵逆的方法, 常见的有雅可比迭代法、Gauss-Seidel 迭代法和共轭梯度法等。这些方法通过迭代更新逼近矩阵的逆, 可以应用于大型稀疏矩阵或特殊结构的矩阵。

- 伪逆:

- 对于不可逆的矩阵, 可以使用伪逆来近似求解。常见的伪逆包括 Moore-Penrose 伪逆和广义逆, 它们可以在某些情况下代替矩阵的真实逆。



2. 矩阵运算

• 2.5 矩阵求逆 (Inverse)

- 应用：颜色空间变换

- 常见的颜色空间

- RGB (适用于表示色彩, 但不适用于获取图像)
 - YIQ (Y为亮度; IQ为色调的两个分量, I从橙色变为青色, Q从紫色到黄绿色)
 - YUV (彩色电视, Y为亮度, UV为色差, 构成彩色的两个分量, YUV三个量分离)

$$\begin{bmatrix} Y \\ U \\ V \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.147 & -0.289 & 0.436 \\ 0.615 & -0.515 & -0.100 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.147 & -0.289 & 0.436 \\ 0.615 & -0.515 & -0.100 \end{bmatrix}^{-1} \begin{bmatrix} Y \\ U \\ V \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1.140 \\ 1 & -0.395 & -0.581 \\ 1 & 2.032 & 0 \end{bmatrix}^{-1} \begin{bmatrix} Y \\ U \\ V \end{bmatrix}$$



2. 矩阵运算

• 2.5 矩阵求逆 (Inverse)

- 线性回归 (Linear Regression, LR)
 - 基于LR的房价预测

No. of Rooms (x0)	Size (Square feet) (x1)	Year Built (x2)	No. of Floors (x3)	Price (y)
2	1434	2010	1	8500
3	1534	2019	2	9600
2	962	1996	3	25880
...	...	...	...	...
...	...	...	...	...

$$\theta = \arg \min_{\theta} \|y - X\theta\|_2^2$$

Where,

x_i = features of a house

y = target variable

$$x = \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix} \quad \& \quad \theta = \begin{pmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \theta_3 \end{pmatrix}$$

$$\theta = (X^T X)^{-1} X^T y$$

Where

X^T = Transpose of X

y = Prediction

$$x = \begin{pmatrix} 2 & 1434 & 1 \\ 3 & 1534 & 2 \\ 2 & 962 & 3 \end{pmatrix} \quad y = \begin{pmatrix} 8500 \\ 9600 \\ 25880 \end{pmatrix}$$

```
import numpy as np
```

```
x = np.array([[2, 1834, 1],[3, 1534, 2],[2, 962, 3]])# Target or Price
```

```
y = [8500, 9600, 258800]
```

Calculation of theta's value:

```
theta = np.dot(inverse_of_multi_transpose_x_to_x,  
multiplication_transposed_x_y)theta
```

```
array([-1.32997548e+05,  6.57907908e+01,  1.53834785e+05])
```



2. 矩阵运算

• 2.6 伴随矩阵(Adjugate Matrix)

- 伴随矩阵是与原矩阵相关的重要概念，通常用于求解矩阵的逆

• 设矩阵 $A = \begin{pmatrix} a_{11} & a_{12} & \text{L} & a_{1n} \\ a_{21} & a_{22} & \text{L} & a_{2n} \\ \text{M} & \text{M} & & \text{M} \\ a_{n1} & a_{n2} & \text{L} & a_{nn} \end{pmatrix}$ 中元素 a_{ij} 的代数余子式 A_{ij} ,

$$A_{ij} = (-1)^{i+j} \det(A_{ij}).$$

$$A^* = \begin{pmatrix} A_{11} & A_{21} & \text{L} & A_{n1} \\ A_{12} & A_{22} & \text{L} & A_{n2} \\ \text{M} & \text{M} & & \text{M} \\ A_{1n} & A_{2n} & \text{L} & A_{nn} \end{pmatrix}$$

称为A的伴随矩阵。

注：伴随矩阵不是把代数余子式放回原位，而是放回原位之后还要做一个整体转置！



2. 矩阵运算

• 2.6 伴随矩阵

例9: 求二阶方阵的伴随矩阵.

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}$$

解 $A_{11} = a_{22}, A_{22} = a_{11}, A_{12} = -a_{21},$
 $A_{21} = -a_{12},$

$$\Rightarrow A^* = \begin{bmatrix} a_{22} & -a_{21} \\ -a_{12} & a_{11} \end{bmatrix}^T = \begin{bmatrix} a_{22} & -a_{12} \\ -a_{21} & a_{11} \end{bmatrix}$$

二阶可逆矩阵求伴随矩阵：主对角线换位置，副对角线变符号！



2. 矩阵运算

• 2.6 伴随矩阵

- 定理: $AA^* = A^*A = |A|E$.

证明:

$$\begin{aligned} AA^* &= \begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & & \vdots \\ a_{n1} & \cdots & a_{nn} \end{pmatrix} \begin{pmatrix} A_{11} & \cdots & A_{n1} \\ \vdots & & \vdots \\ A_{1n} & \cdots & A_{nn} \end{pmatrix} \\ &= \begin{pmatrix} a_{11}A_{11} + \cdots + a_{1n}A_{1n} & \cdots & a_{11}A_{n1} + \cdots + a_{1n}A_{nn} \\ \vdots & & \vdots \\ a_{n1}A_{11} + \cdots + a_{nn}A_{1n} & \cdots & a_{n1}A_{n1} + \cdots + a_{nn}A_{nn} \end{pmatrix}. \end{aligned}$$

由第行列式展开定理:

$$\begin{aligned} a_{11}A_{11} + \cdots + a_{1n}A_{1n} &= |A| \\ a_{11}A_{n1} + \cdots + a_{1n}A_{nn} &= 0 \end{aligned} \Rightarrow AA^* = \begin{pmatrix} |A| & & 0 \\ & |A| & \\ 0 & & \ddots \\ & & & |A| \end{pmatrix} = |A|E \Rightarrow A^*A = |A|E.$$



2. 矩阵运算

• 2.6 伴随矩阵

- 定理：矩阵A可逆充分必要条件是 $|A| \neq 0$,

$$\text{且当 } |A| \neq 0 \text{ 时 } A^{-1} = \frac{1}{|A|} A^*.$$

证明:

必要性：设A可逆，则 $AA^{-1} = E$ ；两边取行列式有：

$$|A||A^{-1}| = |E| = 1 \neq 0, \Rightarrow |A| \neq 0.$$

充分性：设 $|A| \neq 0$ ，则 $AA^* = A^*A = |A|E$.

$$\Rightarrow A\left(\frac{1}{|A|}A^*\right) = \left(\frac{1}{|A|}A^*\right)A = E. \Rightarrow A^{-1} = \frac{1}{|A|}A^*.$$



2. 矩阵运算

• 2.6 伴随矩阵

- 定理：若方阵 A 、 B 有 $AB = E$ ，则 A 、 B 均可逆.

证明： 由于 $AB = E$

$$|AB| = |A||B| = |E| = 1,$$

$$\Rightarrow |A| \neq 0, |B| \neq 0,$$

$$\Rightarrow A, B \text{ 均可逆.}$$



2. 矩阵运算

• 2.6 伴随矩阵

- 总结： 以下4种情况等价
 - A 是满秩矩阵
 - A 是非奇异矩阵 ($Ax = 0$ 只有0解)
 - A 可逆
 - $|A| \neq 0$



2. 矩阵运算

• 2.6 伴随矩阵

例： $A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ ， A 是否可逆，求其逆矩阵。

解：

$$|A| = \begin{vmatrix} 1 & 2 \\ 3 & 4 \end{vmatrix} = 4 - 6 = -2 \neq 0,$$

→ A 可逆

$$A^{-1} = \frac{1}{|A|} A^* = -\frac{1}{2} \begin{pmatrix} 4 & -2 \\ -3 & 1 \end{pmatrix} = \begin{pmatrix} -2 & 1 \\ \frac{3}{2} & -\frac{1}{2} \end{pmatrix}.$$



2. 矩阵运算

• 2.6 伴随矩阵

例：方程可写成矩阵形式 $Ax = b$, 求 x 。

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 2 & 1 \\ 3 & 4 & 3 \end{pmatrix}, b = \begin{pmatrix} 2 \\ -1 \\ 4 \end{pmatrix}, x = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}.$$

解：由于 $|A| = 2 \neq 0$, 故 A 可逆, 因此 $x = A^{-1}b$,

$$A_{11} = \begin{vmatrix} 2 & 1 \\ 4 & 3 \end{vmatrix} = 2, \quad A_{12} = -\begin{vmatrix} 2 & 1 \\ 3 & 3 \end{vmatrix} = -3,$$

$$A_{13} = \begin{vmatrix} 2 & 2 \\ 3 & 4 \end{vmatrix} = 2, \quad A_{21} = -\begin{vmatrix} 2 & 3 \\ 4 & 3 \end{vmatrix} = 6,$$

$$A_{22} = \begin{vmatrix} 1 & 3 \\ 3 & 3 \end{vmatrix} = -6, \quad A_{23} = -\begin{vmatrix} 1 & 2 \\ 3 & 4 \end{vmatrix} = 2, \quad A_{31} = \begin{vmatrix} 2 & 3 \\ 2 & 1 \end{vmatrix} = -4,$$

$$A_{32} = -\begin{vmatrix} 1 & 3 \\ 2 & 1 \end{vmatrix} = 5, \quad A_{33} = \begin{vmatrix} 1 & 2 \\ 2 & 2 \end{vmatrix} = -2,$$



2. 矩阵运算

• 2.6 伴随矩阵

例：方程可写成矩阵形式 $Ax = b$, 求 x 。

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 2 & 1 \\ 3 & 4 & 3 \end{pmatrix}, b = \begin{pmatrix} 2 \\ -1 \\ 4 \end{pmatrix}, x = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}.$$

$$\Rightarrow A^{-1} = \frac{1}{|A|} A^* = \frac{1}{2} \begin{pmatrix} 2 & 6 & -4 \\ -3 & -6 & 5 \\ 2 & 2 & -2 \end{pmatrix},$$

$$\Rightarrow x = \frac{1}{2} \begin{pmatrix} 2 & 6 & -4 \\ -3 & -6 & 5 \\ 2 & 2 & -2 \end{pmatrix} \begin{pmatrix} 2 \\ -1 \\ 4 \end{pmatrix} = \begin{pmatrix} -9 \\ 10 \\ -3 \end{pmatrix}.$$



2. 矩阵运算

• 2.6 伴随矩阵

例: $A = \begin{pmatrix} 2 & 1 & 1 \\ 2 & 6 & 4 \\ 2 & 1 & 3 \end{pmatrix}$, $AB = A + B$, 求 $A + B$.

解: 由于 $AB = A + B$, $(A - E)B = A$

$$|A - E| = \begin{vmatrix} 1 & 1 & 1 \\ 2 & 5 & 4 \\ 2 & 1 & 2 \end{vmatrix} = 2 \neq 0,$$

$$B = (A - E)^{-1}A. \quad (A - E)^* = \begin{pmatrix} 6 & -1 & -1 \\ 4 & 0 & -2 \\ -8 & 1 & 3 \end{pmatrix}.$$

$$(A - E)^{-1} = \frac{1}{2}(A - E)^* = \frac{1}{2} \begin{pmatrix} 6 & -1 & -1 \\ 4 & 0 & -2 \\ -8 & 1 & 3 \end{pmatrix},$$

$$B = \frac{1}{2} \begin{pmatrix} 6 & -1 & -1 \\ 4 & 0 & -2 \\ -8 & 1 & 3 \end{pmatrix} \begin{pmatrix} 2 & 1 & 1 \\ 2 & 6 & 4 \\ 2 & 1 & 3 \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 8 & -1 & -1 \\ 4 & 2 & -2 \\ -8 & 1 & 5 \end{pmatrix}.$$

$$A + B = \begin{pmatrix} 6 & \frac{1}{2} & \frac{1}{2} \\ 4 & 7 & 3 \\ -2 & \frac{3}{2} & \frac{11}{2} \end{pmatrix}.$$

